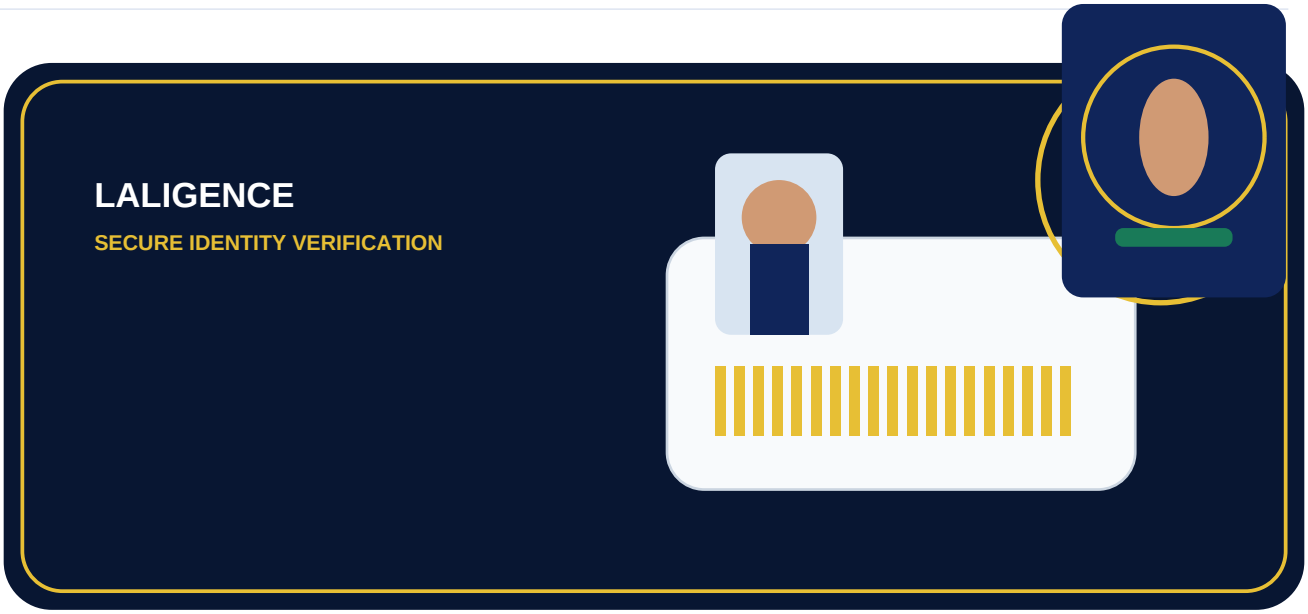


LALIGENCE

A start-to-finish guide for colleagues testing and explaining the LALIGENCE eKYC prototype. It covers the user workflow, the backend checks, Face ID enrollment, payment/login use cases, model training, datasets, and deployment responsibilities.

Internal guidebook

Passport eKYC with liveness, fraud checks, and Face ID login

Audience	Product, engineering, QA, risk, and demo reviewers
Version	Guide generated 2026-05-14
Important	Use sample or redacted identity documents only in public demos.

1. PRODUCT OVERVIEW

What the system does

LALIGENCE is a web-first, mobile-ready eKYC prototype for passport proofing. It guides a user through document capture, active liveness, hand gesture challenge, selfie face match, passive liveness, and a final risk decision. After a successful verification, the system can enroll a returning-user Face ID profile for future login or payment confirmation.

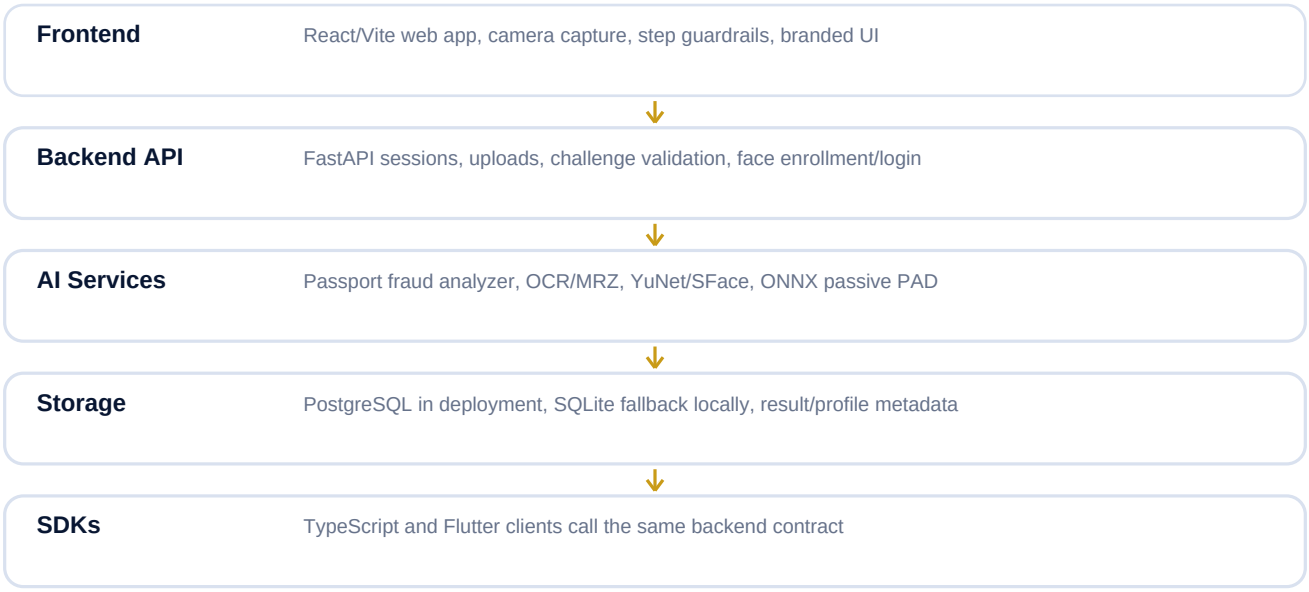
Core promise

- Verify passport evidence with OCR, MRZ checks, quality checks, and fraud risk signals.
- Require active human actions before accepting face liveness.
- Require three randomized hand gestures inside a moving challenge area.
- Compare the selfie against the passport portrait and run passive anti-spoofing.
- Store only verification results, verified profile fields, and biometric templates for enrolled users.

What this prototype is not yet

- It is NIST IAL2-aligned as a workflow, not a certified identity proofing service.
- It is not production-ready without stronger privacy, audit, retention, encryption, and admin access controls.
- It does not train models inside the live product. Training happens offline, then exported models are plugged into backend adapters.

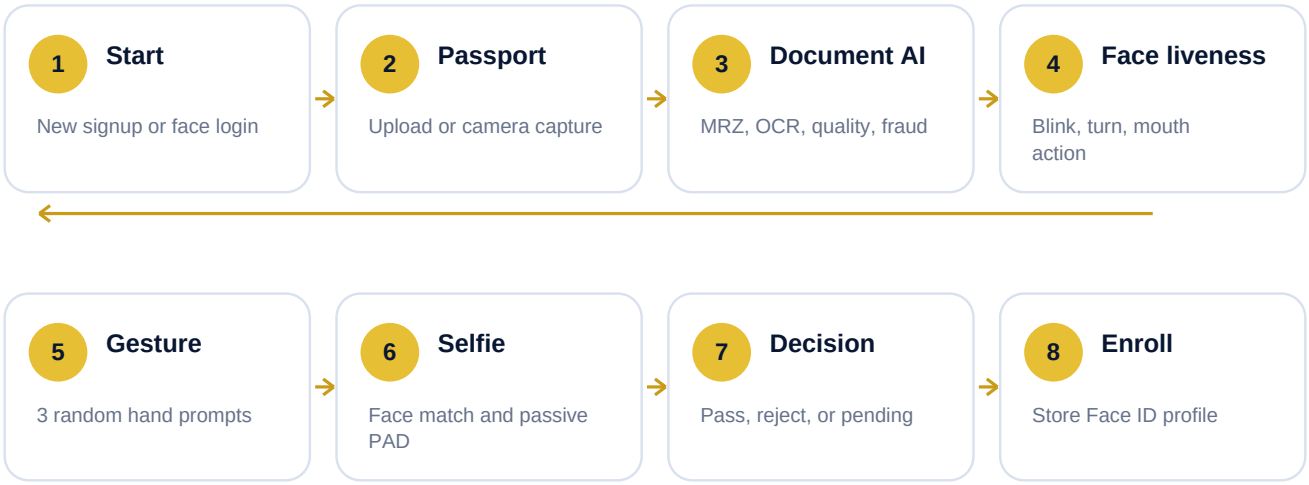
System map



2. END-TO-END WORKFLOW

From first signup to verified Face ID

The user journey is intentionally strict: each step unlocks only after the previous evidence passes. A rejected passport blocks liveness; failed active liveness blocks gesture and selfie; failed selfie blocks enrollment.



Step	User action	Backend decision
1	New user enters user_id or returning user chooses Face Pay login.	Creates a verification session or starts face-login matching.
2	Uploads/captures passport.	Checks file type, size, document quality, MRZ/OCR, fraud, and passport portrait face extraction.
3	Performs randomized face movement prompts.	Backend verifies captured frame has one live-looking face and no high passive replay risk.
4	Performs three random hand gestures inside the circle.	Marks gesture challenge complete only after required prompts pass.
5	Captures selfie.	Runs selfie quality, passive liveness, face extraction, and SFace match against passport portrait.
6	Reviews result.	Decision is passed, rejected, or pending with reason codes.
7	Enrolls Face ID after pass.	Stores one active face_id per user_id and one verified profile per passport number.

3. PASSPORT UPLOAD

What happens when a document is uploaded

The passport step is the gatekeeper. It must confirm that the evidence looks like a passport, contains readable passport data, has a usable portrait, and does not show strong signs of editing, screen replay, or poor capture quality.

Checks performed

- Content type and maximum file size.
- Image quality: resolution, brightness, contrast, blur, sharpness, and glare.
- Document likeness: rectangular document structure and passport-like layout cues.
- OCR/MRZ: TD3 MRZ extraction, field parsing, and check-digit validation.
- Fraud risk: recapture artifacts, compression anomalies, text overlay/editing, crop replacement, and portrait-substitution cues.
- Passport portrait face detection and in-memory embedding extraction for later selfie comparison.

Important rejection examples

- Non-document image, such as a random photo or object.
- Unreadable or invalid MRZ.
- Expired passport once expiry is extracted.
- Severe blur, glare, or cropped evidence.
- High document fraud risk or high tamper risk.
- No usable face found in the passport portrait area.

Output	Meaning
OCR fields	Name, passport number, nationality, birth date, expiry, MRZ validity, and raw extracted fields.
Scores	Image quality, document likeness, recapture risk, tamper risk, fraud risk.
Signals	Explainable reason codes such as MRZ_NOT_READ, BLUR_DETECTED, DOCUMENT_FACE_NOT_FOUND, or DOCUMENT_FRAUD_RISK_HIGH.
Session embedding	The passport portrait template is kept only in the in-memory verification session for selfie matching.

4. LIVENESS AND GESTURE CHALLENGE

How the system reduces replay risk

Active liveness and hand gestures make the flow harder to pass with only a static photo. The current web test version focuses on correct action detection plus backend evidence validation. Stronger production replay defense requires calibrated PAD data and challenge-response timing controls.

Active face liveness

- The session randomly selects three prompts: turn left, turn right, blink, or open mouth.
- The browser detects movement locally to guide the user.
- The backend receives challenge evidence through /active-liveness.
- The backend checks exactly one usable face and runs passive PAD against the captured frame.

Hand gesture challenge

- Three hand prompts are randomly selected from one, two, three, four, five, OK, thumb down, and I love you.
- The challenge circle appears at random positions and requires the hand to be inside it.
- The frontend uses gesture detection for a smooth test experience.
- A production version should store signed challenge evidence or server-verify gesture frames.

Replay warning

Frontend landmark motion alone is not enough. A phone video can mimic blink and head movement, so the backend must inspect captured evidence before accepting a challenge.

Why random prompts matter

Random prompts make pre-recorded attacks harder because the attacker must know the action sequence and timing before the session starts.

5. SELFIE, FACE MATCH, AND FACE ID

How a verified profile is created

The selfie step confirms that the live person matches the passport portrait. A passed verification can then create a profile for returning login and future payment confirmation.

Component	Current implementation	Why it matters
Face detection	OpenCV YuNet ONNX.	Finds the user face in passport, selfie, and login images.
Face matching	OpenCV SFace ONNX embeddings and similarity comparison.	Rejects selfie evidence that does not match the passport portrait.
Passive PAD	facenox/Silent-Face-Anti-Spoofing ONNX plus full-frame screen/phone heuristics.	Catches screen replay, paper/photo attacks, and suspicious display frames.
Enrollment	POST /api/verifications/{session_id}/enroll-face after decision passed.	Creates one active face_id per user_id and one verified profile per passport number.
Face login	POST /api/face-login with fresh live selfie.	Matches returning users and returns verified profile fields for login or payment.

Stored after enrollment

- face_id, user_id, active flag, verification session ID.
- Full name, first name, last name, age, date of birth, nationality.
- Passport number and passport expiry.
- Face template vector, model metadata, enrollment and last-login timestamps.

Not stored by policy

- Raw passport image.
- Raw selfie image.
- Raw liveness or gesture video.
- Unprotected biometric data for public production use.

6. PAYMENT/BANKING USE CASE

Face Pay transfer flow

The app can act like a banking demo: new users verify once, enroll Face ID, and later authorize transfer actions with a live face check instead of repeating full passport proofing.

Flow	Action	Security check
Signup	User starts as a new customer and enters the app's real user_id.	Full eKYC: passport, liveness, gesture, selfie, final decision.
Enroll	After pass, backend stores a verified Face ID profile.	One user_id to one active face_id; one passport number to one verified profile.
Transfer	Returning user taps Face Pay transfer.	Backend checks live face, passive PAD, and template match.
Approve	If matched, app can display verified profile and continue payment authorization.	Production should add amount limits, device trust, audit logs, and step-up auth for high-risk payments.

Production banking note

Face login is a biometric authentication signal, not a complete payment authorization system by itself. For real banking use, combine it with account authentication, device binding, transaction risk scoring, consent, audit logs, and regulatory review.

7. HOW THE AI IS TRAINED AND TESTED

Offline training, online inference

The live backend should not train models on user uploads. Training and calibration happen offline with consented, labeled data. The trained model is exported to ONNX or another runtime format, then plugged into backend adapters for inference.

Model area	Classes to collect	Key metrics
Passport fraud	Genuine passport, crop/face replacement, text/field edit, text overlay, expired, missing fields, blur/glare, screen/print recapture, non-document.	False accept rate, false reject rate, precision/recall by fraud class, MRZ read rate.
Passive face PAD	Live face, screen replay, phone display, printed photo, paper/cardboard mask, low-light live, glasses live.	APCER, BPCER, ACER, spoof false accept, live false reject.
Active liveness	Correct human actions, wrong actions, still image, replayed video, multiple faces, poor lighting.	Challenge pass rate, replay failure rate, action confusion matrix.
Gesture	Each supported hand gesture, wrong gesture, hand outside circle, low light, background hands.	Gesture accuracy, per-class recall, false pass for wrong prompt.
Face match	Same-person passport/selfie pairs, different-person pairs, age/lighting variation, document photo variation.	FAR, FRR, ROC/AUC, threshold at acceptable risk.

Recommended training process

- Define labels and reject reasons before collecting data.
- Split by person/document, not randomly by image, to avoid leakage.
- Train on train set, tune thresholds on validation set, report only test-set metrics.
- Export model to ONNX and wrap it in a backend service adapter.
- Calibrate thresholds for the business risk: low false accept for security, controlled false reject for usability.

Current local test data

- IDNet-style generated genuine and fraud examples under test_dataset.
- 1,200 synthetic hard passport-scene images for upload stress testing.
- Selfie spoof installer for screen, paper, and larger AxonData frame samples.
- MIDV-2020 support after the licensed dataset is downloaded locally.

8. HOW QA SHOULD TEST IT

Test scripts and expected outputs

Testing should measure both security and usability. A good detector must reject attacks, but it must also avoid rejecting too many real users under normal phone and laptop conditions.

Test	Command
Backend compile	PYTHONPATH=backend .venv/bin/python -m compileall backend/app
Frontend build	npm run build
Document fraud benchmark	PYTHONPATH=backend .venv/bin/python backend/scripts/evaluate_document_fraud.py --dataset test_dataset
Upload route benchmark	PYTHONPATH=backend .venv/bin/python backend/scripts/evaluate_document_upload.py --dataset test_dataset --only-folder genuine_synthetic_us_passports_hard
Selfie spoof benchmark	PYTHONPATH=backend .venv/bin/python backend/scripts/evaluate_selfie_spoof.py --dataset test_dataset/selfie_spoof
List demo profiles	curl http://localhost:8000/api/profiles | python -m json.tool

Security tests

- Upload edited passport photo and confirm rejection.
- Upload non-document image and confirm rejection.
- Try phone screen replay for active liveness and selfie.
- Try a different person's selfie against a passport.
- Try multiple faces in selfie/login frame.

Usability tests

- Test real passport in daylight, low light, and mild glare.
- Test live selfie with and without glasses.
- Test mobile camera on same network and public HTTPS URL.
- Check mobile layout for start page, document, liveness, gesture, selfie, and result.
- Record false rejects with reason codes and sample conditions.

9. DEPLOYMENT AND OPERATIONS

How colleagues can test together

The preferred company demo setup is a DigitalOcean Singapore backend with PostgreSQL and Caddy HTTPS, plus a Vercel frontend. This avoids Render sleep and keeps the API closer to Laos and Southeast Asia testers.

Layer	Recommended setup	Why
Backend	DigitalOcean Droplet, Singapore, Docker Compose.	Stable machine, no free-tier sleep, closer region.
Database	PostgreSQL in Docker for demo or managed PostgreSQL for production.	Stores verified profiles and Face ID templates.
TLS/proxy	Caddy with API domain.	Browsers and phones require HTTPS for camera and secure uploads.
Frontend	Vercel static Vite deployment.	Fast static hosting and easy env config.
Mobile SDK	TypeScript SDK and Flutter package call the same API.	Mobile UI stays thin; backend owns risk decisions.

Environment settings to confirm

Set VITE_API_BASE_URL on the frontend. Set DATABASE_URL, LALIGENCE_CORS_ORIGINS, upload size limits, rate limit, and Face ID thresholds on the backend. Keep CORS explicit for hosted demos.

10. PRODUCTION READINESS CHECKLIST

What must be added before real customers

The prototype is useful for internal demos and model evaluation. Production eKYC requires stronger compliance, data protection, observability, and operational controls.

Security and privacy

- Encrypt biometric templates at rest.
- Add authentication and authorization to profile/admin endpoints.
- Add consent capture and clear retention/deletion policy.
- Add audit logs for verification, enrollment, login, and deletion.
- Protect secrets and database backups.
- Run threat modeling for spoofing, replay, account takeover, and insider access.

Compliance and operations

- Review NIST IAL2 requirements formally; do not claim certification without assessment.
- Add monitoring, uptime alerts, and health checks.
- Add automated PostgreSQL backups and restore drills.
- Add load testing for upload and face-login endpoints.
- Create incident response procedure for biometric and identity data exposure.
- Document tester guidance: use sample or redacted documents only.

Model governance

- Version every model and threshold.
- Keep benchmark datasets separate from training data.
- Track false accepts and false rejects by device and lighting condition.
- Add manual review lane for borderline cases.

Key message for colleagues

How to explain the system

LALIGENCE verifies passport evidence, proves the person is live, matches the selfie to the document portrait, gives an explainable decision, and can enroll a Face ID profile for later login or Face Pay transfer.